Gestionale Affitti — Class Diagram di design

Le operazioni dei Service che attraversano porte di persistenza sono asincrone.
Le porte di persistenza restituiscono Promise perché le implementazioni concrete usano pg.
GeneratoreDocumentoContratto e DataCorrenteProvider restano sincroni: nella v1.0 non richiedono I/O asincrono.
Gli id del design sono interi e vengono assegnati dalla persistenza PostgreSQL.
Il contenuto storico del Contratto è una stringa HTML nella v1.0.
I confini HTTP ricevono i rispettivi Service per dependency injection e non costruiscono dipendenze infrastrutturali.
Il composition root è l'unico punto che conosce insieme web, application e implementazioni concrete dell'infrastruttura.
Le implementazioni PostgreSQL realizzano le porte dell'application layer senza introdurre dipendenze da pg nei Service o nel dominio.

Intercetta le richieste /api prima dei controller.
Ispeziona path, query e body tramite SicurezzaInputService.
I pattern sospetti producono HTTP 400 e un log tecnico
con il valore reso sicuro per la registrazione.
La rilevazione euristica non sostituisce validazione
specifica, query parametrizzate o HTML escaping.

Confine HTTP di UC-01: traduce JSON in input applicativi,
invoca il Service e serializza le risposte.
Espone anche l'anteprima HTML della bozza completa.
Le date di calendario attraversano HTTP come YYYY-MM-DD.
Non contiene regole di dominio o query SQL.

Preview applicativa per la visualizzazione.
Alla conferma Contratto e Pagamenti vengono ricaricati
e i valori vengono ricalcolati lato server.

Application model temporaneo identificato da idBozza.
Più bozze possono coesistere nella v1.0 mono-utente;
per un Immobile già registrato è ammessa al massimo
una bozza attiva. Conserva soltanto i dati necessari
a riprendere la procedura. Conserva la tipologia selezionata
con i template necessari alla ripresa, ma non contiene Contratto
generato, documento HTML o Pagamenti definitivi.

L'anteprima costruisce un Contratto transitorio
dalla bozza completa e usa lo stesso generatore
della conferma senza produrre scritture definitive.
Alla conferma il Contratto viene ricostruito e
il documento viene rigenerato dai dati rivalidati.

In UC-01 id e codiceFiscale non vengono modificati.
Il codice fiscale viene normalizzato e validato
strutturalmente, ammettendo l'omocodia.
L'IBAN è opzionale e appartiene alla Persona;
il template può richiederlo al proprietario.
La data di nascita deve corrispondere a un'età
compresa tra 18 e 150 anni rispetto alla data server.
I dati recuperati sono una working copy verificabile
e modificabile prima della conferma definitiva.

Template della tipologia contrattuale.
numArticolo identifica l'articolo logico;
numParte ordina eventuali frammenti dello stesso articolo.
I dati dinamici sono richiesti tramite placeholder {{...}}
nel testo, non tramite convenzioni sui numeri degli articoli.

{ proprietario.codiceFiscale <> inquilino.codiceFiscale }
{ canoneMensile > 0 }
{ giornoPagamento in 1..28 }
{ al calcolato da dal e tipologia.durata prima della registrazione
e poi conservato come parte del periodo storico }
{ dal e al inclusivi }
{ registratoIl assegnato alla conferma tramite DataCorrenteProvider }
Durante l'assemblaggio il contenuto può essere assente.
Il GeneratoreDocumentoContratto produce l'HTML completo e
il Service lo assegna prima della registrazione definitiva.
{ registrazione definitiva: contenuto presente e almeno 1 Pagamento }

{ almeno 1 Articolo template }

{ meseCompetenza in 1..12 }
{ univoco per Contratto + annoCompetenza + meseCompetenza }

Porta di scrittura di UC-02.
I Pagamenti esistenti vengono ricostruiti
con il Contratto tramite ContrattoRepository.

{ pre: il Contratto da registrare possiede
contenuto completo e almeno 1 Pagamento }

In UC-01 il documento usato dall'inquilino
deve avere dataRilascio non successiva e
dataScadenza strettamente successiva alla data
corrente fornita dal DataCorrenteProvider.
La validazione viene ripetuta per anteprima e conferma.

Nazione, CAP, civico, scala e interno sono opzionali:
non sono dati universalmente disponibili né per
l'ubicazione di un Immobile né per la residenza.

Confine HTTP di UC-02: valida identificativi e competenza,
invoca il Service e serializza preview e Pagamento.
Non decide la mensilità, non calcola importi
e non accede direttamente alla persistenza.

Unico punto di assemblaggio delle dipendenze concrete.
Costruisce pool, repository, adapter e provider
e li inietta nei Service dei due casi d'uso.

I template sono testo con placeholder espliciti {{...}}.
I placeholder supportati sono una lista chiusa;
valori e template vengono HTML-escaped.
numArticolo e numParte determinano soltanto l'ordine.

Implementa il confine transazionale di UC-01.
Usa una singola connessione PostgreSQL:
BEGIN -> dati definitivi collegati -> Contratto
-> Pagamenti -> COMMIT; in caso di errore ROLLBACK.
Il cleanup della Bozza resta esterno alla transazione.

Usa la data di calendario dell'orologio del processo
e la normalizza a mezzanotte UTC per il modello date-only.

web
«middleware» SicurezzaInputHttpMiddleware
«service» SicurezzaInputService
+verifica<T>(valore : T, percorso : string) : T
«controller» RegistraContrattoController
+router : Router
«controller» RegistraPagamentoController
+router : Router
«module» compositionRoot
+creaApplicazioneProduzione(): ApplicazioneProduzione
application
model
«application model» PagamentoDaRegistrare
-contrattoId : int
-canoneMensile : decimal
-tipologiaDenominazione : string
-dal : date
-al : date
-annoCompetenza : int
-meseCompetenza : int
-scadenza : date
-importo : decimal
-dovuta : boolean
-tardivo : boolean
«application model» BozzaContratto
-idBozza : int [0..1]
-stepCompletato : int
-nomeDescrizione : string [0..1]
-dal : date [0..1]
-al : date [0..1]
-canoneMensile : decimal [0..1]
-giornoPagamento : int [0..1]
«service» RegistraPagamentoService
+elencaImmobili(): Promise<List<Immobile>>
+elencaInquilini(immobileId : int): Promise<List<Persona>>
+preparaPagamento(immobileId : int, inquilinoId : int): Promise<PagamentoDaRegistrare | null>
+confermaPagamento(contrattoId : int, annoCompetenza : int, meseCompetenza : int): Promise<Pagamento>
«service» RegistraContrattoService
+avvia(): Promise<List<BozzaContratto>>
+elencaImmobili(): Promise<List<Immobile>>
+elencaTipologie(): Promise<List<TipologiaContrattuale>>
+selezionaImmobile(immobileId : int, idBozza : int [0..1]): Promise<BozzaContratto>
+inserisciNuovoImmobile(immobile : Immobile, idBozza : int [0..1]): Promise<BozzaContratto>
+cercaPersona(codiceFiscale : string): Promise<Persona | null>
+cercaPersonaPerBozza(idBozza : int, ruolo : string, codiceFiscale : string): Promise<Persona | null>
+impostaProprietario(idBozza : int, persona : Persona): Promise<BozzaContratto>
+impostaInquilino(idBozza : int, persona : Persona): Promise<BozzaContratto>
+impostaDatiContrattuali(idBozza : int, nomeDescrizione : string, tipologiaId : int, dal : date, canoneMensile : decimal, giornoPagamento : int): Promise<BozzaContratto>
+anteprima(idBozza : int): Promise<string>
+conferma(idBozza : int): Promise<void>
+annulla(idBozza : int): Promise<void>
ports
ContrattoRepository
+trovaPerId(id : int): Promise<Contratto | null>
+trovaPerImmobile(immobileId : int): Promise<List<Contratto>>
+esisteSovrapposizione(immobileId : int, dal : date, al : date): Promise<boolean>
ImmobileRepository
+trovaTutti(): Promise<List<Immobile>>
+trovaPerId(id : int): Promise<Immobile | null>
+trovaPerDatiCatastali(dati : DatiCatastali): Promise<Immobile | null>
+esisteConIndirizzo(indirizzo : Indirizzo): Promise<boolean>
GeneratoreDocumentoContratto
+genera(contratto : Contratto) : string
PagamentoRepository
+salva(contrattoId : int, pagamento : Pagamento): Promise<void>
DataCorrenteProvider
+oggi() : date
TipologiaContrattualeRepository
+trovaTutte(): Promise<List<TipologiaContrattuale>>
+trovaPerIdConArticoli(id : int): Promise<TipologiaContrattuale | null>
PersonaRepository
+trovaPerCodiceFiscale(codiceFiscale : string): Promise<Persona | null>
BozzaContrattoRepository
+elenca(): Promise<List<BozzaContratto>>
+trovaPerId(idBozza : int): Promise<BozzaContratto | null>
+trovaPerImmobileId(immobileId : int): Promise<BozzaContratto | null>
+salva(bozza : BozzaContratto): Promise<BozzaContratto>
+elimina(idBozza : int): Promise<void>
RegistrazioneContrattoPort
+registraDefinitivamente(contratto : Contratto): Promise<void>
domain
TipologiaContrattuale
-id : int [0..1]
-denominazione : string
-durata : int
-rinnovo : int
Articolo
-id : int [0..1]
-numArticolo : int
-numParte : int
-titolo : string
-sottotitolo : string [0..1]
-descrizione : string
Persona
-id : int [0..1]
-nome : string
-cognome : string
-luogoNascita : string
-dataNascita : date
-codiceFiscale : string
-iban : string [0..1]
+normalizzaCodiceFiscale(codiceFiscale : string) : string
+validaDataNascita(dataNascita : date, oggi : date) : void
+aggiornaDatiAnagrafici(nome : string, cognome : string, luogoNascita : string, dataNascita : date) : void
+cambiaResidenza(residenza : Indirizzo) : void
+impostaDocumentoRiconoscimento(documento : DocumentoRiconoscimento) : void
Contratto
-id : int [0..1]
-nomeDescrizione : string
-dal : date
-al : date
-canoneMensile : decimal
+/canoneAnnuale : decimal
-giornoPagamento : int
-registratoIl : date
-contenuto : string [0..1]
+siSovrapponeA(altro : Contratto) : boolean
+calcolaImportoCompetenza(anno : int, mese : int) : decimal
+calcolaScadenzaCompetenza(anno : int, mese : int) : date
+validaCanoneMensile(canoneMensile : decimal) : void
+validaGiornoPagamento(giornoPagamento : int) : void
+periodiSiSovrappongono(dal : date, al : date, altroDal : date, altroAl : date) : boolean
+calcolaDataFine(dal : date, tipologia : TipologiaContrattuale) : date
+impostaContenuto(contenuto : string) : void
+aggiungiPagamento(pagamento : Pagamento) : void
Pagamento
-id : int [0..1]
-annoCompetenza : int
-meseCompetenza : int
-dataPagamento : date
-importo : decimal
Immobile
-id : int [0..1]
-nome : string
DocumentoRiconoscimento
-id : int [0..1]
-tipo : string
-organoEmittente : string
-dataRilascio : date
-dataScadenza : date
-numero : string
+validaRilascioAlla(oggi : date) : void
+validaScadenzaAlla(oggi : date) : void
Indirizzo
-id : int [0..1]
-nazione : string [0..1]
-provincia : string
-comune : string
-cap : string [0..1]
-indirizzo : string
-civico : string [0..1]
-scala : string [0..1]
-interno : string [0..1]
DatiCatastali
-id : int [0..1]
-codiceComunale : string
-foglio : int
-particella : int
-subalterno : int
-categoria : string
-consistenza : decimal
-rendita : decimal
tipologia
proprietario
inquilino
immobile
articoli predefiniti
pagamenti
documento
residenza
oggetto della locazione
ubicazione
dati catastali
1
0..1
1..*
0..*
infrastructure
document
«generator» GeneratoreDocumentoHtmlContratto
+genera(contratto : Contratto): string
persistence
postgres
«repository» PostgresTipologiaContrattualeRepository
«repository» PostgresImmobileRepository
«repository» PostgresContrattoRepository
«repository» PostgresPagamentoRepository
+salva(contrattoId : int, pagamento : Pagamento): Promise<void>
«repository» PostgresPersonaRepository
«repository» PostgresBozzaContrattoRepository
«transactional adapter» PostgresRegistrazioneContratto
+registraDefinitivamente(contratto : Contratto): Promise<void>
«mapper» PostgresValueMapper
«mapper» BozzaContrattoJsonMapper
time
«provider» DataCorrenteSistemaProvider
+oggi(): date